PROJECT EXECUTION & PERFORMANCE REPORT

(MLOps Model Fine-Tuning, Infrastructure Configuration, and Systemic Learnings)

1. Overview & Operational Objectives

This report details the operational implementation, runtime deployment, and system telemetry for MLOps Assignment 2. The core objective of this assignment focuses on adapting, scaling, and stabilizing an end-to-end transformer-based sequence classification pipeline. Originally modeled within localized constraints, the ecosystem was successfully migrated to a high-capacity cloud-hosted infrastructure (Kaggle Sandbox Environments) to support compute-heavy deep learning workloads.

The architecture establishes a rigorous workflow combining Hugging Face's modeling paradigm with enterprise telemetry sync utilities like Weights & Biases (W&B). This facilitates deep, granular logging of loss matrices, model checkpoints, parameter updates, and evaluation metrics across training epochs. To simulate a strict real-world pipeline constraint, the process — from raw dependency provisioning to target artifact generation — was unified to run in a continuous, automated single-pass execution sequence.

2. Model Selection & Tokenization Strategy

2.1 Core Architectural Choice: DistilBERT Cased

For the primary text classification task (multi-genre Goodreads book reviews), the chosen model is **distilbert-base-cased**. DistilBERT represents a distilled, compact variant of the standard BERT model, engineered via knowledge distillation. It retains approximately 97% of BERT's language comprehension capabilities while utilizing 40% fewer parameters, rendering it structurally optimized for resource-bounded or time-constrained infrastructure.

The choice of the 'cased' variant over 'uncased' is intentional and metrics-driven. In complex literary texts, book summaries, and structural reviews, case-sensitive indicators convey substantial context regarding the underlying narrative genre.

2.2 Tokenization Pipeline

Text preprocessing is delegated to the **DistilBertTokenizerFast** class, which implements a Rust-backed tokenization algorithm. This allows for near-instantaneous batch encoding. The operational configurations of the data ingestion layer include:

- **Max Length Truncation:** Fixed at 128 tokens to cap extreme sequence outliers, balancing context preservation against tensor matrix space requirements.
- **Dynamic Border Padding:** Shorter review items are padded dynamically to match matrix dimensions within individual data loader steps, keeping memory foot-print lean.
- **Vocabulary Alignment:** The cased tokenizer explicitly maps out-of-vocabulary elements to sub-word units while respecting original alphabetical capitalization states.

3. Pipeline Workflow Setup & Data Engineering

The experimental framework constructs an isolated, reproducible sequence splitting mechanism to ensure evaluation stability:

- **1.** Stream Extraction & Volume Control: Data streams are pulled dynamically from cloud storage repositories (hosting diverse Goodreads JSON collections). Review payloads are parsed down and capped via random index sampling to an active working slice of 1,000 samples per genre across 8 target classes (yielding an active operational dataset size of 8,000 observations).
- **2.** Train-Test Discretization: The data matrix is strictly divided into training and testing partitions using a randomized stratify vector split. This guarantees that each distinct genre category maintains proportional representation across both training pools and evaluation partitions.
- **3.** Torch Interface Customization: Encoded token vectors and attention matrices are wrapped inside a tailored PyTorch Dataset subclass (MyDataSet). This class structures key/value tokens directly to conform to Hugging Face Trainer input signatures, preventing memory padding leakage across batch tensors.

Note:

Pipeline Structural Integrity: The sequence is fully modularized so that data loading, preprocessing, optimization tracking, and performance visualization can run sequentially without cross-contaminating validation records.

4. Kaggle Infrastructure & Hardware Configuration

Executing deep transformer fine-tuning requires specialized graphics acceleration. The pipeline leverages a remote cloud kernel on Kaggle equipped with dedicated GPU acceleration (NVIDIA T4X2 Tensor Core or equivalent backend runtime). This hardware layer handles multi-dimensional tensor contractions via CUDA acceleration interfaces, decreasing epoch iteration cycles from several hours (on CPU) to a mere 26 minutes total.

4.1 Environment Secrets and API Security Integration

To align with secure MLOps standards, sensitive operational access keys are never hardcoded into the notebook script. Instead, they are bound using Kaggle's internal **UserData Secrets Environment** wrapper. At initialization, the system calls these isolated keys to bind the runtime instance with external hubs:

- • Hugging Face Hub Security: Handled via private access tokens to authorize artifact pulls, verify identity, and manage model downloads safely.
- • Weights & Biases Stream Sync: Handled via W&B API keys, allowing for immediate workspace binding. This automatically pipes hardware consumption logs and loss trends to an offsite telemetry board.

5. Experimental Performance Results

The model was fine-tuned over 3 complete training iterations (epochs) using an AdamW optimizer configured with an initial base learning rate of 5e-5, a batch allocation of 10 samples per device, and 100 linear learning rate warmup steps. The optimization trajectory shows consistent convergence, as recorded in the active experiment history:

Step Count	Training Loss	Validation Loss	Accuracy Score
100	1.9519	1.6834	0.3944 (39.44%)
300	1.3394	1.3554	0.5063 (50.63%)
500	1.2673	1.2686	0.5619 (56.19%)
700	1.0901	1.2112	0.5863 (58.63%)
1100	0.8920	1.1979	0.5869 (58.69%)
1500	0.5702	1.2496	0.5856 (58.56%)
1700	0.5394	1.2704	0.5963 (59.63%)
1920 (Final)	0.5135	1.2801	0.5888 (58.88%)

Final Model Verification Summary: Following execution of the 1,920 evaluation passes, the model stabilized at step 1920, yielding a final evaluation validation loss of 1.2803 with a global classification accuracy score of 58.88%.

6. Core MLOps Insights & Systemic Learnings

The development and operational deployment of this transformer classification architecture yielded critical, real-world engineering insights across distinct areas:

1. Evaluation Gap Analysis: Overfitting & Decision Regularization Dynamics:

Analyzing the fine-tuning history reveals a widening performance gap between training loss and validation loss during the latter half of training. Between steps 700 and 1920, the training loss steadily dropped from 1.0901 down to 0.5135, signaling aggressive semantic absorption of the training text corpus. However, the corresponding validation loss hit a structural inflection minimum at step 1100 (1.1979) before slightly tracking upwards to 1.2801 at termination.

This divergence highlights a classical over-parameterization drift. While the classification accuracy continued to stabilize near 58.88%, future pipeline revisions should incorporate early stopping

parameters or higher weight decay metrics to regularize model weights before validation returns diminish.

2. Data Bottleneck Trades: Balancing Compute Constraints vs. Sample Scale:

Deep sequence networks scale quadratically with token lengths. Restricting text arrays to a maximum boundary of 128 tokens proved essential for preventing out-of-memory errors on the GPU. This constraint allowed for larger batch groupings while maintaining consistent token generation throughput. This highlights the importance of data caching and strategic truncation in production machine learning environments.

3. Telemetry Integration Value: Automated Experiment Tracking Benefits:

Integrating external tracking tools like Weights & Biases eliminated the need for manual performance logging. Live monitoring of metrics made it easy to visualize performance drifts in real time. This automated layer provides essential transparency, making experiment verification highly efficient and audit-ready.

7. Cloud Environment & Artifact Registries

To ensure accessibility, transparency, and artifact integrity across the project lifecycle, all deployment notebooks, fine-tuned model checkpoints, and live telemetry boards have been published to public cloud registries:

- **Kaggle Notebook:** Kaggle Notebook Link
- **Hugging Face:** Hugging Face Profile
- **W&B Dashboard:** [https://wandb.ai/g25ait2113-iitj/huggingface?nw=nwuserg25ait2113)
- **Source Colab:** Google Colab Source "